What is Software? ***

Software is a program that enables a computer to perform a specific task. Software acts as an interface between the user and the computer hardware.

Major Classes of Software

1. System Software

System software manages and controls computer hardware and provides a platform for other software to run.

Examples: Windows, Linux, macOS.

2. Programming Software

Programming software provides tools for developers to create, test, debug, and maintain programs.

Examples: Compiler, Interpreter, Visual Studio.

3. Application Software

Application software is designed to help users perform specific tasks.

Examples: Microsoft Word, Microsoft Excel, Google Chrome, Adobe Photoshop.

Easy Exam Definition

"Software is a collection of programs, instructions, and data that enables a computer to perform specific tasks."

Memory Trick

SPA = System Software + Programming Software + Application Software ✓

Define Software Testing and Software Quality Assurance (SQA). Explain the differences between them with suitable examples.

Software Testing

Definition:
Software testing is a process of checking and evaluating the functionality of a software application with an intent to find whether the developed software met the specified requirements or not and verify that it meets the specified requirements.

Easy Meaning:
Testing checks whether the software works correctly or not.

Example:
Suppose a Calculator App is developed.
The tester enters **10 + 5**.

- Expected Output = **15**
- If the output is **15**, the test passes.
- If the output is **12**, a defect is found.

Software Quality Assurance (SQA)

Definition:
Software Quality Assurance (SQA) is a set of activities to ensure the quality in software engineering processes that ultimately result in quality software products. The activities establish and evaluate the processes that produce products. It involves process-focused action.

Easy Meaning:
SQA focuses on improving the development process to prevent defects.

Example:
An SQA team reviews coding standards, conducts process audits, verifies documentation, and ensures developers follow established development methodologies to reduce the chances of defects.

Difference between Software Testing and Software Quality Assurance (SQA)

Software Testing	Software Quality Assurance (SQA)
Testing finds defects in the software.	SQA prevents defects by improving the development process.
It focuses on the final software product.	It focuses on the software development process.

Performed after or during software development.	Performed throughout the entire software development life cycle (SDLC).
Detects bugs and verifies functionality.	Ensures quality standards and best practices are followed
Usually performed by testers or QA engineers.	Ensures quality standards and best practices are followed.
Usually performed by testers or QA engineers.	Performed by SQA engineers, quality managers, and the entire development team.
Objective is to verify that the product works correctly.	Objective is to ensure the process produces high-quality software.
It is a product-oriented activity.	It is a process-oriented activity.
Goal: Find bugs.	Goal: Prevent bugs and improve quality.

Define Software Testing and Debugging. Explain the differences between them with suitable examples.

Software Testing

Definition:
Software testing is a process of checking and evaluating the functionality of a software application with an intent to find whether the developed software met the specified requirements or not and verify that it meets the specified requirements.

Example:
Suppose a Calculator App is developed.
The tester enters **10 + 5**.

- Expected Output = **15**

- Actual Output = **12**

The tester reports this as a **bug**.

Debugging

Definition:
Debugging is the process of identifying, analyzing, and fixing the defects (bugs) found during software testing.

Easy Meaning:
Debugging is the process of fixing the bugs found during testing.

Example:
The developer checks the calculator code and finds that subtraction (−) was used instead of addition (+). After changing it to addition (+), the program correctly displays **15**.

Difference between Software Testing and Debugging

Software Testing	Debugging
Finds defects (bugs) in the software.	Fixes the defects (bugs) found during testing.
Performed by testers (QA).	Performed by developers.
Identifies where the problem exists.	Identifies the cause and removes the problem.
Focuses on detecting errors.	Focuses on correcting errors.
Done before debugging.	Done after testing identifies the bug.
Does not modify the source code.	Requires changes to the source code to fix the issue.
Starts after or during software development.	Starts after a defect has been identified through testing or reported by users.

According to **ANSI/IEEE 1059**, software testing is the process of analyzing software to identify differences between expected and actual results (defects) and to evaluate its features.

Quality Software:

Quality software refers to a software which is reasonably bug or defect free, is delivered in time and within the specified budget, meets the requirements and/or expectations, and is maintainable.

Types of Software Quality

1. Functional Quality

It measures how well the software performs the functions required by users according to the specifications.

Example: A banking app correctly transfers money and shows account balance.

2. Structural Quality

It refers to the non-functional aspects of software such as maintainability, reliability, robustness, and efficiency.

Example: Software is easy to modify and continues to work properly even under heavy load.

Software Quality Control (SQC)

Software Quality Control (SQC) is a set of activities to ensure the quality in software products. These activities focus on determining the defects in the actual products produced. It involves product-focused action.

Comparison: Software Products vs Industrial Products (Easy Version)

Characteristic	Software Products	Industrial Products
Complexity	Has **millions of operational possibilities**	Has only **thousands of operational options**

Characteristic	Software Products	Industrial Products
Visibility	Software is **invisible**, so defects cannot be seen directly	Industrial products are **visible**, so defects can be seen easily
Defect Detection	Defects are found mainly during **development/testing phase**	Defects can be found in **all phases (design, production planning, manufacturing)**

One-Line Memory:
Software is complex and invisible, so testing is harder; industrial products are visible and easier to inspect. ✓

How we will get quality software?

1. Clear Requirements

Just as we need to know how many floors or rooms a house should have before building it, we must clearly understand and document the client's requirements before development starts. If the requirements are wrong, the software will not be useful, no matter how fast it is.

2. Solid Architecture & Design

Just as a house needs a proper blueprint from an architect, software needs a strong architecture and database design before development begins. This helps the system remain stable even when the number of users grows.

3. Following SDLC & Standards

Developers should follow a standard **Software Development Life Cycle (SDLC)** process (such as Agile) and maintain clean coding standards. Poorly written code becomes difficult to maintain and update later.

4. Rigorous Testing

Just as a house is inspected after construction to ensure electricity and water work properly, software must be thoroughly tested through **Unit Testing, Integration Testing, and System Testing** to find and fix bugs.

5. User Involvement

Just as homeowners regularly check the progress of their house construction, users or clients should stay involved throughout development and provide feedback to ensure the software meets their needs.

6. Proper Maintenance

The job is not finished after delivering the software. Regular maintenance is necessary to fix issues, improve performance, and provide updates when needed.

Short Exam Answer

Quality software can be achieved through clear requirements, solid design, following SDLC standards, rigorous testing, user involvement, and proper maintenance. ✅

Memory Trick:

Requirements → Design → Coding Standards → Testing → User Feedback → Maintenance

Difference between Software Fault, Error and Failure with Appropriate Example and Code

1. Software Fault (Bug/Defect)

Definition:
A software fault is a mistake or defect in the source code made by the programmer.

Example:
A programmer writes subtraction (−) instead of addition (+) in a calculator program.

Code:

```
#include <stdio.h>

int main() {
    int a = 10, b = 5;
    int sum = a - b;    // Fault: should be a + b
    printf("%d", sum);
```

```
    return 0;
}
```

Explanation:
The statement `a - b` is the **Fault** because the programmer wrote incorrect code.

2. Software Error

Definition:
A software error is an incorrect result produced during program execution due to a fault.

Example:

Input:

```
a = 10
b = 5
```

Expected Output:

```
15
```

Actual Output:

```
5
```

Explanation:
Because of the fault (`a - b`), the program produces **5 instead of 15**. This incorrect result is called an **Error**.

3. Software Failure

Definition:
A software failure occurs when the software cannot perform and its required functionality do not working correctly.

Example:
A user tries to add **10 + 5**, but the calculator displays **5** instead of **15**. The software fails to provide the correct result.

Explanation:
The user experiences the wrong behavior of the software. This is called a **Failure**.

Difference Table

No.	Fault (Bug/Defect)	Error	Failure
1	Mistake in the code.	Wrong internal calculation or state.	Wrong output or system behavior.
2	Created by the developer.	Caused when a fault is executed.	Happens when the error reaches the user.
3	Exists in the source code.	Exists during program execution.	Visible to the user.
4	It is the **cause**.	It is the **effect of the fault**.	It is the **result of the error**.
5	Found during code review or testing.	Detected while the program is running.	Seen during testing or by end users.
6	Does not always produce a problem.	May or may not lead to failure.	Always shows incorrect behavior.
7	Example: `a/0` instead of `a/b`.	Division by zero occurs during execution.	Program crashes with an error message.
8	Fixed by correcting the code.	Removed when the fault is fixed.	Disappears after the error is corrected.
9	Also called a **bug** or **defect**.	Also called an **incorrect state**.	Also called a **software malfunction**.
10	Occurs before execution.	Occurs during execution.	Observed after execution as incorrect output.

Fault	Error	Failure
Mistake in the source code.	Incorrect result during execution.	Software cannot perform the required function correctly.
Created by the programmer.	Caused by a fault.	Caused by an error.
Exists in the code.	Appears during execution.	Visible to the user.

Define Software Verification and Software Validation. Explain the differences between them with suitable examples.

Software Verification

Definition:
Software Verification is the process of checking whether the software is being developed **correctly according to the specified requirements**.

Easy Meaning:

"Are we building the product right?"

Example:

Suppose a software requirement says that the **Login page must contain Username and Password fields**.

During verification, we check whether the developer has correctly implemented these two fields according to the design and requirements.

Software Validation

Definition:
Software Validation is the process of checking whether the **final software meets the user's needs and intended use**.

Easy Meaning:

"Are we building the right product?"

Example:

After the software is completed, the user logs in successfully using the Login page and can use all the required features correctly. This confirms that the software satisfies the user's needs.

Difference between Verification and Validation

Verification	Validation
Checks whether the software is built **correctly**.	Checks whether the **correct software** is built.
Performed during software development.	Performed after software development is completed.
Focuses on documents, design, and code.	Focuses on the final software product.
Finds defects early.	Confirms that the software meets user requirements.
Question: "Are we building the product right?"	**Question:** "Are we building the right product?"

Defective Software

Defective software means software that contains **errors, bugs, or faults**. Almost every software we develop has some defects, and it is difficult to make it 100% perfect.

☞ It is very hard to predict all future problems, so software may still contain defects even after testing.

Sources of Problems in Software

- **Requirements Definition:** Erroneous, incomplete, or inconsistent requirements.
- **Design:** Fundamental design flaws in the software.
- **Implementation:** Mistakes in chip fabrication, wiring, programming faults, malicious code.
- **Support Systems:** Poor programming languages, faulty compilers and debuggers, misleading development tools.
- **Inadequate Testing of Software:** Incomplete testing, poor verification, mistakes in debugging.
- **Evolution:** Sloppy maintenance or redevelopment, new bugs while fixing old ones, increasing system complexity over time.

মনে রাখার ট্রিক:

☞ **R D I S T E**

Adverse Effects of Faulty Software

- **Communications:** Loss or corruption of communication media, non-delivery of data.
- **Space Applications:** Loss of lives, launch delays.
- **Defense and Warfare:** Misidentification of friend or foe.
- **Transportation:** Deaths, delays, sudden acceleration, inability to brake.
- **Safety-critical Applications:** Death, injuries.
- **Electric Power:** Death, injuries, power outages, long-term health hazards (radiation).
- **Money Management:** Fraud, privacy violation, shutdown of stock exchanges and banks, negative interest rates.
- **Control of Elections:** Wrong results (intentional or unintentional).
- **Control of Jails:** Technology-aided escape attempts, accidental release of inmates, failure of software-controlled locks.
- **Law Enforcement:** False arrests and wrongful imprisonments.

Short Technique:

C S D T S E M E J L

ত্রুটিপূর্ণ (Faulty) সফটওয়্যারের ক্ষতিকর প্রভাব (Adverse Effects of Faulty Software)

- **যোগাযোগ ব্যবস্থা (Communications):** যোগাযোগ মাধ্যমের তথ্য হারিয়ে যাওয়া বা নষ্ট হওয়া, এবং তথ্য (Data) সঠিকভাবে পৌঁছাতে ব্যর্থ হওয়া।
- **মহাকাশ কার্যক্রম (Space Applications):** মানুষের প্রাণহানি এবং রকেট/মিশন উৎক্ষেপণে বিলম্ব।
- **প্রতিরক্ষা ও যুদ্ধ (Defense and Warfare):** শত্রু ও মিত্রকে ভুলভাবে শনাক্ত করা।
- **পরিবহন ব্যবস্থা (Transportation):** প্রাণহানি, যাতায়াতে বিলম্ব, গাড়ির হঠাৎ গতি বেড়ে যাওয়া এবং **ব্রেক (Brake) কাজ না করা**।
- **নিরাপত্তা-সংক্রান্ত গুরুত্বপূর্ণ অ্যাপ্লিকেশন (Safety-critical Applications):** মৃত্যু ও গুরুতর আহত হওয়ার ঘটনা।
- **বিদ্যুৎ ব্যবস্থা (Electric Power):** মৃত্যু, আহত হওয়া, বিদ্যুৎ বিভ্রাট এবং দীর্ঘমেয়াদি স্বাস্থ্যঝুঁকি (যেমন: বিকিরণ/Radiation)।

- **অর্থ ও ব্যাংকিং ব্যবস্থাপনা (Money Management):** জালিয়াতি (Fraud), ব্যক্তিগত তথ্যের গোপনীয়তা লঙ্ঘন (Privacy Violation), শেয়ারবাজার ও ব্যাংক বন্ধ হয়ে যাওয়া এবং ভুলভাবে ঋণাত্মক সুদের হার (Negative Interest Rate) প্রয়োগ।
- **নির্বাচন নিয়ন্ত্রণ (Control of Elections):** ইচ্ছাকৃত বা অনিচ্ছাকৃতভাবে ভুল নির্বাচনী ফলাফল প্রকাশ।
- **কারাগার নিয়ন্ত্রণ (Control of Jails):** প্রযুক্তির সাহায্যে পালানোর চেষ্টা, ভুলবশত বন্দিদের মুক্তি এবং সফটওয়্যার-নিয়ন্ত্রিত তালা (Lock) কাজ না করা।
- **আইন প্রয়োগকারী সংস্থা (Law Enforcement):** নির্দোষ ব্যক্তিকে ভুলভাবে গ্রেপ্তার করা এবং অন্যায়ভাবে কারাবন্দি করা।

Just Mone rakhar jonno:

- **Communications** → Data loss

- **Space Applications** → Life loss, launch delay
- **Defense & Warfare** → Wrong target identification
- **Transportation** → Accidents, brake failure
- **Safety-critical Applications** → Death, injuries
- **Electric Power** → Power outage, radiation risk
- **Money Management** → Fraud, privacy violation
- **Control of Elections** → Wrong election results
- **Control of Jails** → Prisoner escape/release

- **Law Enforcement** → False arrest

Testing Goals Based on Test Process Maturity Level

This model shows how the understanding and purpose of software testing have evolved over time.

Level 0 Thinking: Testing = Debugging

Main Idea:

Testing and debugging are considered the same.

Easy Explanation:

- Finding and fixing bugs is considered testing.

- No distinction is made between software failures and programming mistakes.
- It does not help develop reliable or safe software.
- This is commonly how undergraduate Computer Science students are initially taught.

Exam Point:

Testing is considered the same as debugging and does not help build reliable software.

Level 1 Thinking: Show Correctness

Main Idea:

The purpose of testing is to prove that the software is correct.

Easy Explanation:

- Complete correctness is impossible to prove.
- If no failures are found, we cannot be sure whether:
  - The software is good, or
  - The tests were poor.
- Test engineers have:
  - No strict goal
  - No real stopping rule
  - No formal testing technique
- Test managers have little control.
- This is often what hardware engineers expect.

Exam Point:

The goal is to prove software correctness, but complete correctness can never be guaranteed.

Level 2 Thinking: Show Failures

Main Idea:

The purpose of testing is to find failures and defects.

Easy Explanation:

- Testers focus on finding bugs and failures.
- Testing becomes a negative activity.
- Testers and developers may develop an adversarial relationship.
- If no failures are found, we still cannot guarantee that the software is perfect.
- Most software companies work at this level.

Exam Point:

The purpose of testing is to find failures, which may create conflict between testers and developers.

Level 3 Thinking: Reduce Risk

Main Idea:

The purpose of testing is to reduce the risk of using the software.

Easy Explanation:

- Testing can only show the presence of failures, not their absence.
- Every software system involves some risk.
- Risks may be:
    - Small and unimportant, or
    - Serious and catastrophic.
- Testers and developers work together to reduce risk.
- A few advanced software companies follow this approach.

Exam Point:

Testing helps reduce the risk of using software through cooperation between testers and developers.

Level 4 Thinking: Improve Quality

Main Idea:

Testing is a mental discipline that improves software quality.

Easy Explanation:

- Testing is one way to improve software quality.
- Test engineers can become technical leaders of a project.
- Their main responsibility is:
    - Measuring software quality
    - Improving software quality
- Their expertise helps developers create better software.
- This is how traditional engineering disciplines work.

Exam Point:

Testing is a quality improvement discipline that helps develop high-quality software.

Easy Summary Table

Level	Goal
Level 0	Testing = Debugging
Level 1	Show Correctness
Level 2	Find Failures
Level 3	Reduce Risk
Level 4	Improve Software Quality

পর্ব ১: Software Testing Model (টেস্টিং এর ডায়াগ্রাম)

এই ডায়াগ্রামে একটি সফটওয়্যার টেস্ট করার প্রক্রিয়া বা ফ্লো দেখানো হয়েছে। ধরুন আপনি একটি ক্যালকুলেটর অ্যাপ বানিয়েছেন এবং সেটি টেস্ট করছেন:

- **Test Data (টেস্ট ডেটা):** এটি হলো আপনার দেওয়া ইনপুট। যেমন: আপনি টেস্ট করার জন্য ক্যালকুলেটরে ২ + ২ দিলেন।
- **Software under Test (সফটওয়্যার):** এটি হলো আপনার আসল কোড বা সফটওয়্যার (ক্যালকুলেটর অ্যাপটি) যাকে আপনি টেস্ট করছেন।
- **Output (আউটপুট):** আপনার সফটওয়্যার প্রসেস করার পর যে রেজাল্ট বা ফলাফলটি দেয়। যেমন: অ্যাপটি হিসাব করে ৪ (বা কোডে কোনো ত্রুটি থাকলে ৫) আউটপুট দিলো।
- **Expected Output (প্রত্যাশিত আউটপুট):** টেস্ট করার আগেই আপনি জানেন যে সঠিক রেজাল্ট কী আসা উচিত। এখানে প্রত্যাশিত আউটপুট হলো ৪।

- **Oracle (ওরাকল):** এটি হলো সেই ব্যবস্থা বা মেকানিজম যা আসল **Output** এবং **Expected Output**-কে তুলনা (compare) করে। ওরাকল যদি দেখে দুটোই হুবহু মিলে গেছে, তাহলে টেস্ট "Pass" (পাস), আর না মিললে "Fail" (ফেল)।

Part 1: Software Testing Model

This diagram shows the **basic flow of software testing**. Let's understand it with a simple example of a **calculator app**.

1. Test Data

Test data means the **input we give for testing**.

☞ Example:
We enter **2 + 2** in the calculator.

2. Software under Test (SUT)

This is the **actual software or program** that we are testing.

☞ Example:
The calculator application (your code).

3. Output (Actual Result)

This is the **result given by the software after processing the input**.

☞ Example:
The calculator shows **4** (or sometimes wrong output like 5 if there is a bug).

4. Expected Output

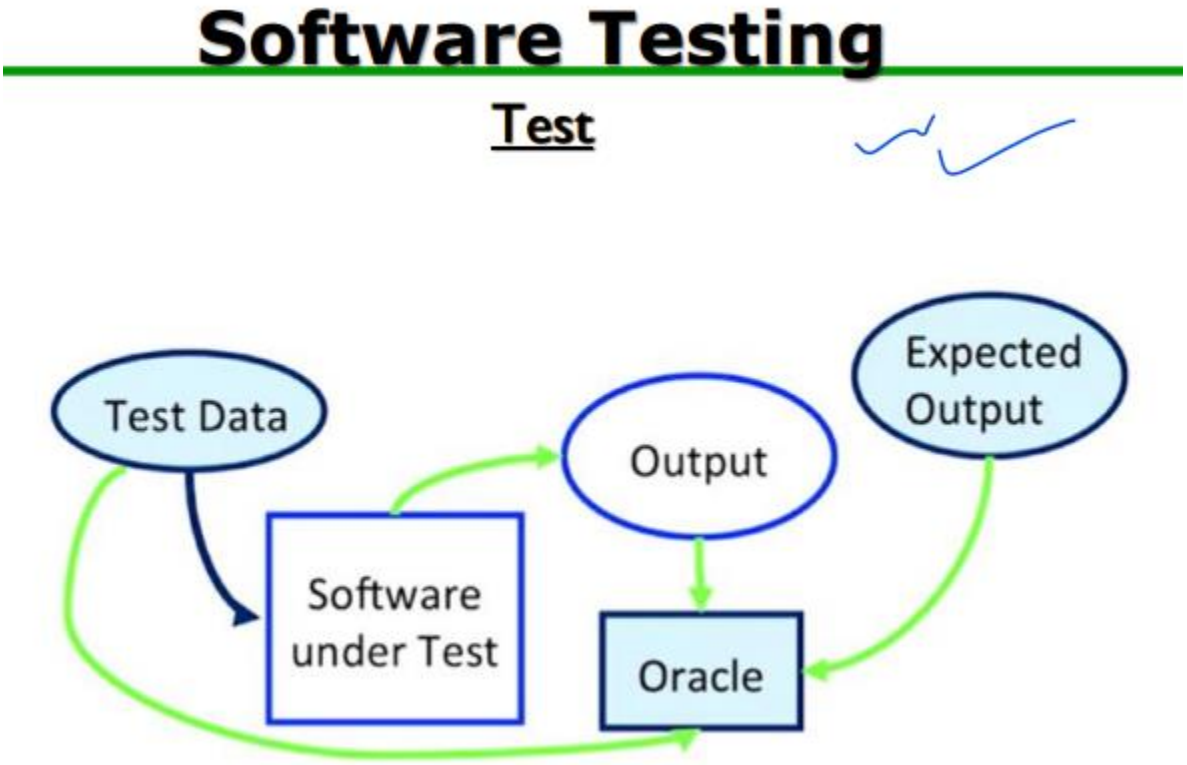
This is the **correct result that we expect before testing**.

☞ Example:
For **2 + 2**, the expected output is **4**.

5. Oracle

Oracle is the **system or mechanism that compares actual output with expected output**.

- If both match → **Test Pass**
- If they do not match → **Test Fail**



Part 2: Anatomy of a Test

Every standard software test has **4 main steps**. We can understand it easily by comparing it with a **science lab experiment**.

1. Setup (Preparation)

Before starting the test, we prepare everything needed for testing.

- Load test data
- Open database connections
- Set variables and environment

☞ **Easy meaning:**
Prepare the system for testing.

 Example comparison:
Like arranging all equipment before starting a lab experiment.

2. Invocation (Execution / Running)

This is the step where we actually **run the code or function**.

☞ **Easy meaning:**
Execute the software or function that we want to test.

 Example comparison:
Like starting the experiment and mixing chemicals.

3. Assessment (Checking Result)

In this step, we **compare the actual output with the expected output**.

☞ This is usually done by an **Oracle**.

☞ **Easy meaning:**
Check whether the result is correct or not.

 Example comparison:
Like comparing experiment results with textbook answers.

4. Teardown (Cleanup)

After testing is finished, we **clean everything and restore the system**.

- Close database connections
- Delete test files
- Reset environment

☞ **Easy meaning:**
Bring the system back to its original state.

 Example comparison:
Like cleaning the lab after the experiment.

Easy Summary

☞ **Setup → Invocation → Assessment → Teardown**

Fault & Failure Model (RIPR)

RIPR Model (Very Easy English)

To find a **failure** in a software program, **4 conditions** must happen:

1. Reachability

The test must **reach the faulty code**.

Example:

```
if(age > 18)
{
   // bug is here
}
```

If `age = 15`, this code does not run.

➞ Fault is **not reached**.

2. Infection

The fault must make the **program state incorrect**.

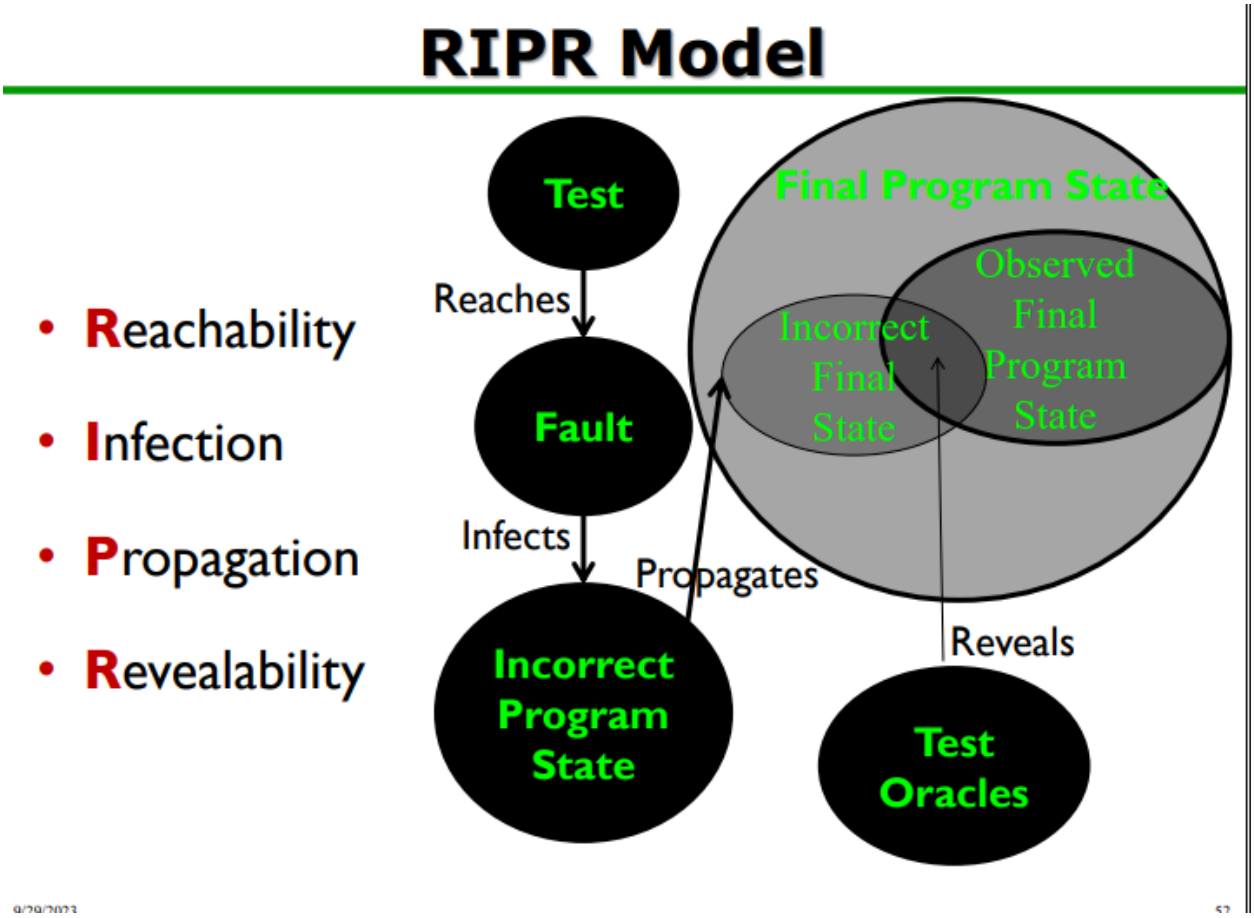
Example:

```
total = price - benefit;  // should be +
```

If `price = 100` and benefit `= 20`,
result becomes `80` instead of `120`.

➞ The program state is wrong.

➞ Infection occurs.



3. Propagation

The incorrect state must **affect the final output**.

Example:

```
total = price - benefit; // wrong
total = 120;             // corrected later
```

The error happened, but it did not reach the output.

➞ No propagation.

4. Revealability

The tester must **notice the wrong output**.

Example:
Expected Output = 120

Actual Output = 80

If the tester does not check the result, the failure is not detected.

➞ Error exists, but it is not revealed.

R → I → P → R
☞ **Reach → Infect → Propagate → Reveal**

V-Model (Easy Exam Version)

Easy Idea of V-Model

In the **V-Model**, every **development phase** has a corresponding **testing phase**.

Left Side = Development
Right Side = Testing

So, whatever is designed on the left side is verified by testing on the right side.

Easy Formula

Development Phase	Testing Phase
Requirements Analysis	Acceptance Testing
Architectural Design	System Testing
Subsystem Design	Integration Testing
Detailed Design	Module Testing
Implementation (Coding)	Unit Testing

1. Requirements Analysis ↔ Acceptance Testing

English

During **Requirements Analysis**, customer needs and requirements are gathered.

Acceptance Testing checks whether the completed software satisfies the customer's requirements and expectations.

It answers the question:

"Does the software do what the user wants?"

Acceptance testing is usually performed by users or domain experts.

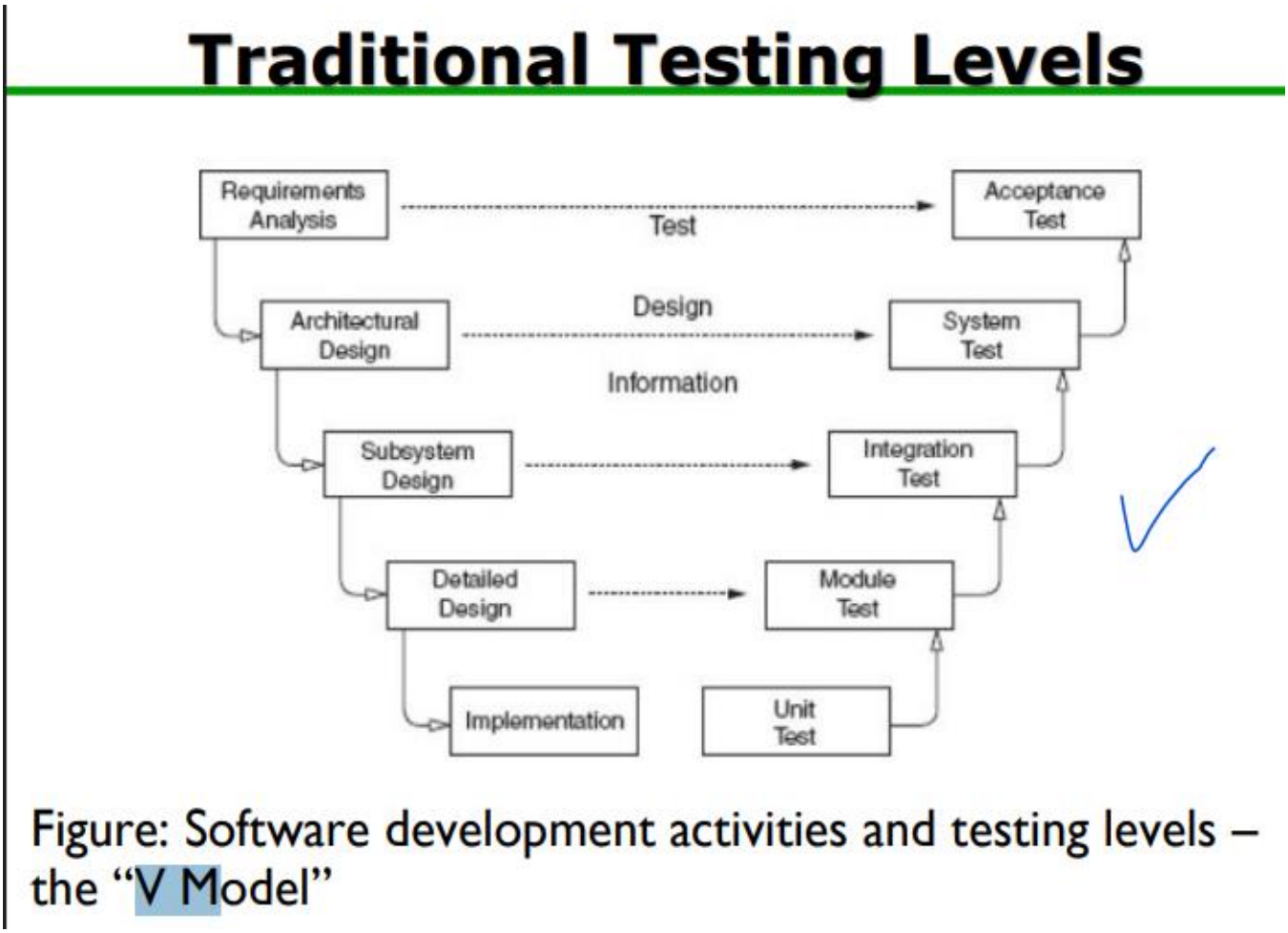


Figure: Software development activities and testing levels – the "V Model"

2. Architectural Design ↔ System Testing

English

During **Architectural Design**, the overall system structure, components, and connections are designed.

System Testing verifies whether the complete integrated system works according to the specifications.

It answers:

"Does the whole system work correctly?"

System testing usually focuses on design and specification issues.

3. Subsystem Design ↔ Integration Testing

During **Subsystem Design**, the structure and behavior of subsystems are defined.

Integration Testing checks whether different modules communicate correctly and whether their interfaces are compatible.

It answers:

"Do the modules work together correctly?"

Integration testing assumes that individual modules already work properly.

4. Detailed Design ↔ Module Testing

English

During **Detailed Design**, the internal structure and behavior of individual modules are designed.

Module Testing checks whether each module works correctly in isolation.

It also verifies interactions among units and data structures inside the module.

Usually performed by developers.

5. Implementation ↔ Unit Testing

English

Implementation is the coding phase where actual program code is written.

Unit Testing is the lowest level of testing that checks individual functions, methods, or procedures.

It answers:

"Does this single unit work correctly?"

Usually performed by programmers.

MDTD (Model-Driven Test Design)

Description

1. Domain Knowledge & Requirements

Collect and analyze the system requirements and understand the application domain. This step ensures that testing is based on the actual business requirements.

2. Test Design (Model Creation)

Create test models and generate test cases based on the system requirements and application behavior.

3. Test Automation (Test Scripts)

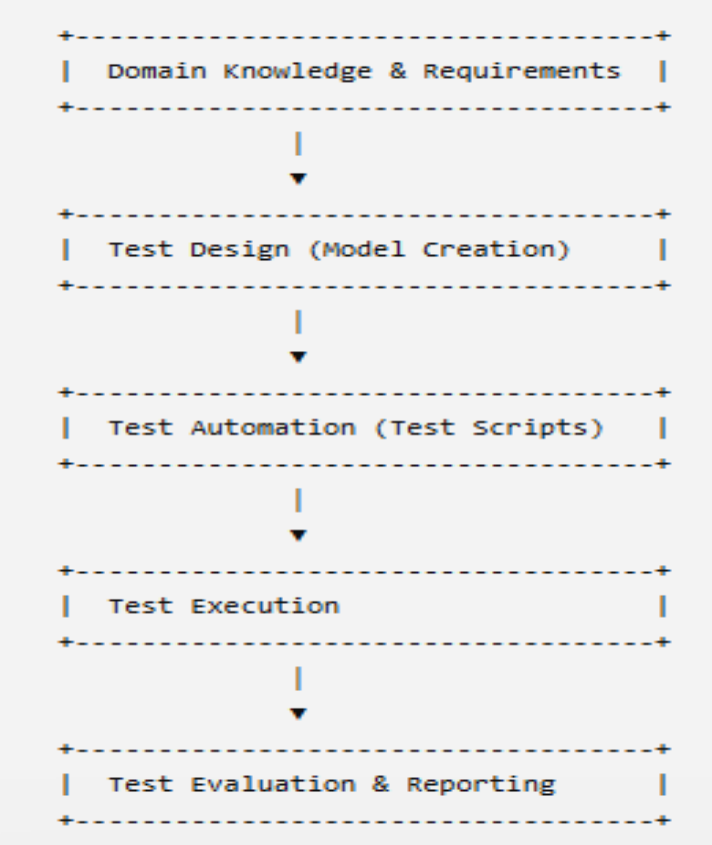
Convert the designed test cases into automated test scripts using suitable automation tools.

4. Test Execution

Execute the automated test scripts on the software and record the actual results.

5. Test Evaluation & Reporting

Analyze the test results, identify defects, compare actual and expected outputs, and prepare the final test report.


Domain Knowledge & Requirements
Test Design (Model Creation)
Test Automation (Test Scripts)
Test Execution
Test Evaluation & Reporting